

Analyse de données de la toxicité du paracétamol sur le foie

Rabah ACHOUR, Mohand Arezki BRANECHI

A) Motivation

On souhaite étudier l'influence du paracétamol sur la toxicité hépatique à partir d'un jeu de données transcriptomiques sur des rats. On dispose de données d'expression génique (3116 gènes) mesurées sur 64 rats exposés à différentes doses de paracétamol et à différents temps après exposition. L'objectif est d'identifier les gènes dont l'expression est significativement affectée par la dose de paracétamol, la périodicité du traitement, ou l'interaction entre ces deux facteurs.

```
rm(list=ls())  
library(MultiVarSel)
```

```
## Le chargement a nécessité le package : Matrix
```

```
## Le chargement a nécessité le package : glmnet
```

```
## Loaded glmnet 4.1-10
```

```
## Le chargement a nécessité le package : ggplot2
```

```
library(ggplot2)
```

B) Exploration des données

1) Examen brut des données

```
load("liver_data.Rdata")  
ls()
```

```
## [1] "liver.toxicity"
```

On a un objet `liver.toxicity`. Regardons la structure globale de l'objet

```
summary(liver.toxicity)
```

```
##           Length Class      Mode
## gene       3116  data.frame list
## clinic      10  data.frame list
## treatment    4  data.frame list
## gene.ID      2  data.frame list
```

On a 4 éléments dans l'objet `liver.toxicity`. Regardons chacun d'eux.

```
treatment
```

```
table1 = liver.toxicity$treatment
str(table1)
```

```
## 'data.frame': 64 obs. of 4 variables:
## $ Animal.Number : int 202 203 204 206 208 209 210 211 212 213 ...
## $ Treatment.Group: Factor w/ 16 levels "1500MG/K18 HR",...: 16 16 13 13 14 14 15 15 15 16 ...
## $ Dose.Group : num 50 50 50 50 50 50 50 50 50 50 ...
## $ Time.Group : int 6 6 18 18 24 24 48 48 48 6 ...
```

```
table1[1:3, ]
```

```
##      Animal.Number Treatment.Group Dose.Group Time.Group
## 202           202      50 MG/KG6 HR          50          6
## 203           203      50 MG/KG6 HR          50          6
## 204           204      50 MG/KG18 HR          50         18
```

La colonne `Treatment.Group` contient la même information que `Dose.Group × Time.Group`, enlevons-la.

```
table1$Treatment.Group <- NULL
table1[1:3, ]
```

```
##      Animal.Number Dose.Group Time.Group
## 202           202          50          6
## 203           203          50          6
## 204           204          50         18
```

Examinons la répartition des deux facteurs `Dose.Group` et `Time.Group`.

```
table(table1$Dose.Group)
```

```
##
## 50 150 1500 2000
## 16 16 16 16
```

```
table(table1$Time.Group)
```

```
##
##  6 18 24 48
## 16 16 16 16
```

```
table(table1$Dose.Group, table1$Time.Group)
```

```
##
##      6 18 24 48
##  50   4  4  4  4
## 150   4  4  4  4
## 1500  4  4  4  4
## 2000  4  4  4  4
```

La distribution des échantillons est parfaitement équilibrée.

Chaque niveau de *Dose.Group* (50, 150, 1500, 2000) contient 16 échantillons.

Chaque niveau de *Time.Group* (6, 18, 24, 48 heures) contient également 16 échantillons.

Le tableau croisé montre que pour chaque combinaison dose-temps, il y a exactement 4 échantillons.

gene

```
table2 = liver.toxicity$gene
dim(table2)
```

```
## [1] 64 3116
```

```
table2[1:3, 1:5]
```

```
##      A_43_P14555 A_43_P22290 A_43_P20792 A_43_P21286 A_43_P12995
## ID202      0.05169    -0.08120     0.00617     0.00003     0.00676
## ID203      0.01548     0.17515     0.04845     0.04730     0.08869
## ID204     -0.01509    -0.04790    -0.02313    -0.02269    -0.07799
```

clinic

```
table3 = liver.toxicity$clinic
str(table3)
```

```
## 'data.frame': 64 obs. of 10 variables:
## $ BUN.mg.dL. : num 12 15 17 19 20 17 16 14 12 11 ...
## $ Creat.mg.dL. : num 0.6 0.7 0.8 0.7 0.7 0.7 0.7 0.6 0.7 0.7 ...
## $ TP.g.dL. : num 7.3 7.5 7.6 7.5 7.8 7.7 7.6 7.4 7.5 7.3 ...
## $ ALB.g.dL. : num 4.9 5 5 5 5.1 5.1 5.1 4.9 5.1 4.9 ...
## $ ALT.IU.L. : int 73 84 54 369 59 57 67 57 54 97 ...
## $ SDH.IU.L. : int 23 23 14 81 16 10 21 19 20 26 ...
## $ AST.IU.L. : int 87 110 129 450 75 80 93 90 76 135 ...
## $ ALP.IU.L. : int 320 325 381 401 359 335 324 309 317 251 ...
## $ TBA.umol.L. : int 9 8 4 13 3 13 8 11 6 7 ...
## $ Cholesterol.mg.dL.: int 85 87 84 82 85 88 82 92 86 77 ...
```

```
table3[1:3, ]
```

```
##      BUN.mg.dL. Creat.mg.dL. TP.g.dL. ALB.g.dL. ALT.IU.L. SDH.IU.L. AST.IU.L.
## 202          12          0.6      7.3      4.9         73         23         87
## 203          15          0.7      7.5      5.0         84         23        110
## 204          17          0.8      7.6      5.0         54         14        129
##      ALP.IU.L. TBA.umol.L. Cholesterol.mg.dL.
## 202          320           9                85
## 203          325           8                87
## 204          381           4                84
```

gene.ID

```
table4 = liver.toxicity$gene.ID
str(table4)
```

```
## 'data.frame': 3116 obs. of 2 variables:
## $ geneBank : Factor w/ 2397 levels "", "AA800287", ...: 1 1 1315 434 1901 106 1665 1773 1 1193 ...
## $ gene.title: Factor w/ 2079 levels "", "1-acylglycerol-3-phosphate 0-acyltransferase 5 (lysophospha
```

```
table4[1:3, ]
```

```
##      geneBank      gene.title
## A_43_P14555
## A_43_P22290
## A_43_P20792 NM_001109576 tRNA splicing endonuclease 54 homolog (S. cerevisiae)
```

On a donc 4 dataframes dans l'objet `liver.toxicity`

- **gene** : données d'expression génique
 $\mathbf{G} \in \mathbb{R}^{64 \times 3116}$
 64 échantillons $\times$ 3116 gènes (valeurs numériques).
- **clinic** : variables cliniques
 $\mathbf{C} \in \mathbb{R}^{64 \times 10}$
 64 échantillons $\times$ 10 mesures cliniques (numériques et entières).
- **treatment** : groupes de traitement
 $\mathbf{T} \in \mathbb{R}^{64 \times 3}$
 64 échantillons $\times$ 3 descripteurs (numéro d'animal, dose [numérique], temps [entier]).
- **gene.ID** : identifiants des gènes
 $\mathbf{I} \in \mathbb{R}^{3116 \times 2}$
 3116 gènes $\times$ 2 champs d'annotation (geneBank, gene.title en facteurs).

2) Valeurs manquantes

```
which(is.na(table1))
```

```
## integer(0)
```

```
which(is.na(table2))
```

```
## integer(0)
```

```
which(is.na(table3))
```

```
## integer(0)
```

On n'a pas de valeurs manquantes. Concentrons nous sur les données `gene` et `treatment`.

3) Visualisation des données de gènes

Les données d'expression génique sont de haute dimension. Pour une visualisation de la variabilité, on peut utiliser une analyse en composantes principales (ACP).

```
pca_samples <- prcomp(table2, scale. = TRUE)
```

Regardons la variance expliquée par chaque composante principale.

```
var_explained <- pca_samples$sdev^2 / sum(pca_samples$sdev^2)
round(var_explained[1:10], 3)
```

```
## [1] 0.281 0.149 0.080 0.054 0.046 0.039 0.035 0.025 0.024 0.018
```

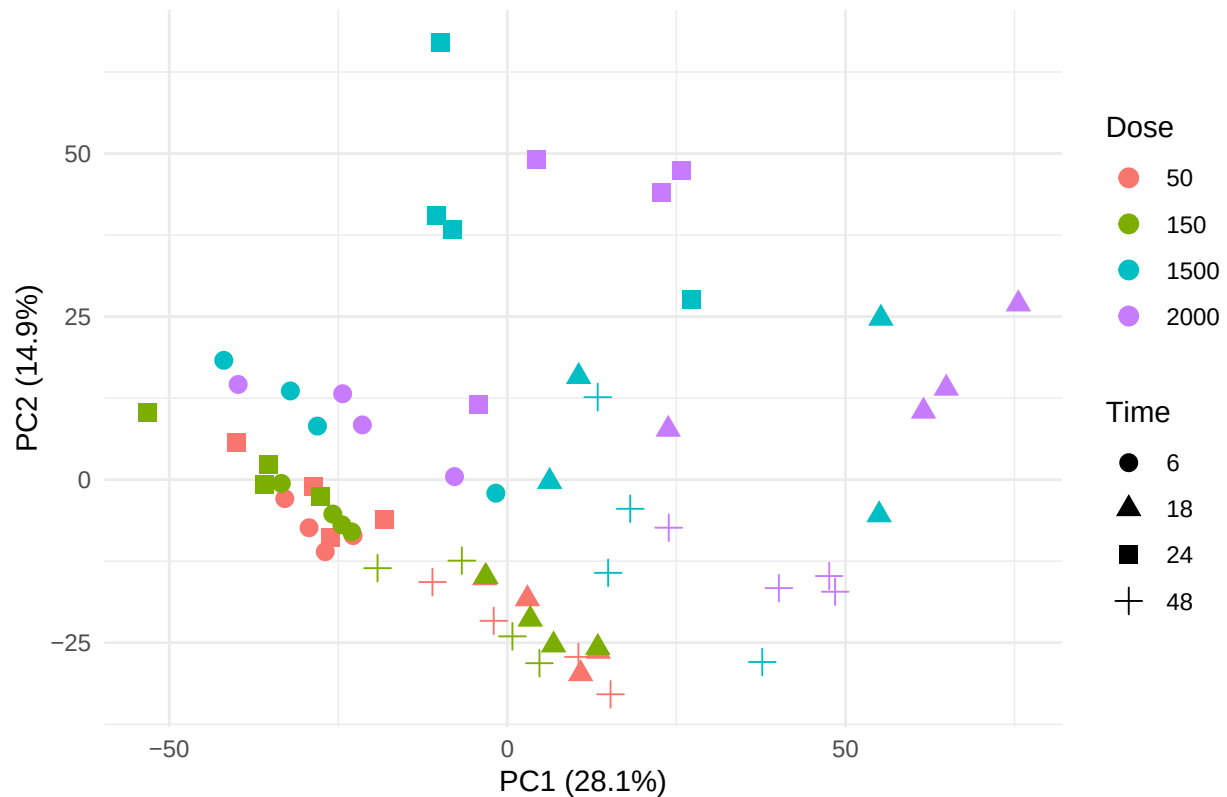
Les deux premières composantes principales expliquent une part significative de la variance totale : PC1 capture 28.1 % et PC2 14.9 %, soit 43 % à elles deux. Cela indique que l'expression génique présente une structure non aléatoire.

Projection des échantillons avec coloration et formes par facteurs Dose et Périodicité

```
df_pca <- data.frame(PC1 = pca_samples$x[,1],
                     PC2 = pca_samples$x[,2],
                     Dose = factor(table1$Dose.Group),
                     Time = factor(table1$Time.Group))

ggplot(df_pca, aes(x = PC1, y = PC2, color = Dose, shape = Time)) +
  geom_point(size = 3) +
  labs(title = "ACP des échantillons selon dose et temps",
       x = paste0("PC1 (", round(var_explained[1]*100,1), "%)"),
       y = paste0("PC2 (", round(var_explained[2]*100,1), "%)")) +
  theme_minimal()
```

ACP des échantillons selon dose et temps



On observe une séparation partielle des échantillons selon la dose et la périodicité, suggérant que ces facteurs influencent l'expression génique. Cette ACP justifie une analyse plus approfondie pour quantifier ces effets.

4) Réduction de la dimensionnalité

Pour alléger les calculs de la sélection de variables, on peut réduire la dimensionnalité en sélectionnant les gènes les plus variables. Le choix de 1500 gènes vient d'un calcul empirique basé sur la RAM disponible (15 Go).

```
gene_importance <- rowSums(pca_samples$rotation[, 1:10]^2)
k <- 1500
selected_genes <- order(gene_importance, decreasing = TRUE)[1:k]
table2_reduced <- table2[, selected_genes]
dim(table2_reduced)
```

```
## [1] 64 1500
```

C) Modélisation linéaire multivariée à deux facteurs

1) Intuition de la modélisation

Le paracétamol est connu pour provoquer une toxicité hépatique à forte dose. Dans notre jeu de données, nous avons deux facteurs expérimentaux :

- **Dose.Group** : la quantité de paracétamol administrée aux rats.
- **Time.Group** : la périodicité du traitement.

Notre hypothèse est que :

- L'expression de certains gènes hépatiques **varie en fonction de la dose** administrée.
- L'expression peut aussi **évoluer selon le temps** après exposition, reflétant la dynamique des lésions hépatiques.
- Il peut exister une **interaction dose $\times$ temps** : par exemple, une forte dose souvent pourrait induire une réponse transcriptionnelle massive, alors qu'une dose plus faible bien étalée pourrait entraîner une modulation plus modérée.

Cette intuition justifie l'utilisation, d'un **modèle linéaire multivarié** pour tester :

- l'effet principal de la dose,
- l'effet principal de la périodicité,
- et leur interaction sur l'expression des gènes.

Remarque : Nous utilisons un modèle multivarié (régression multivariée) plutôt qu'une MANOVA classique, car la matrice de covariance des résidus est de rang inférieur à 64 et n'est donc pas inversible lorsque le nombre de variables (3116 gènes) est très supérieur au nombre d'observations (64). Le modèle linéaire multivarié est la formulation adaptée dans ce cas.

2) Construction de la matrice de design X

Pour ajuster un modèle linéaire multivarié, on construit la matrice de design $X \in \mathbb{R}^{64 \times p}$ qui encode les facteurs expérimentaux *Dose.Group*, *Time.Group* et leur interaction.

Le modèle multivarié s'écrit :

$$Y = X\beta + \varepsilon$$

où :

- $Y \in \mathbb{R}^{64 \times 1500}$ contient les expressions des 1500 gènes,
- X contient les colonnes indicatrices (contrastes) associées à la dose, au temps et à l'interaction,
- $\beta \in \mathbb{R}^{p \times 1500}$ est la matrice des coefficients estimés pour tous les gènes simultanément,
- ε est la matrice des résidus.

Dans notre cas, la matrice X contient :

- une colonne d'intercept,
- trois colonnes pour les contrastes de dose (4 niveaux $\rightarrow$ 3 degrés de liberté),
- trois colonnes pour les contrastes de temps (4 niveaux $\rightarrow$ 3 degrés de liberté),
- neuf colonnes pour les interactions dose $\times$ temps (3×3).

Ainsi,

$$p = 1 + 3 + 3 + 9 = 16.$$

donc

$$X \in \mathbb{R}^{64 \times 16}.$$

```
table_XY <- data.frame(table1[-1], table2_reduced)
table_XY[1:3, 1:10]
```

```
##      Dose.Group Time.Group A_43_P17008 A_43_P19369 A_43_P17618 A_42_P726191
## 202          50          6   -0.00184   0.03701   0.01749   0.04866
## 203          50          6    0.06408   0.04200   0.07975   0.04893
## 204          50         18   -0.00740   -0.00032   -0.00690   0.09739
##      A_43_P19193 A_42_P815757 A_43_P19591 A_43_P10900
## 202    0.00361    0.08862   -0.02615   -0.00965
## 203    0.04925    0.54198    0.11964    0.08473
## 204   -0.03760    0.02901    0.06576   -0.00089
```

Matrice Y et X Modélisation par un modèle linéaire multivarié avec interaction Dose x Temps

```
Y <- as.matrix(table2_reduced)
Y[1:3, 1:5]
```

```
##      A_43_P17008 A_43_P19369 A_43_P17618 A_42_P726191 A_43_P19193
## ID202   -0.00184   0.03701   0.01749   0.04866   0.00361
## ID203    0.06408   0.04200   0.07975   0.04893   0.04925
## ID204   -0.00740   -0.00032   -0.00690   0.09739   -0.03760
```

```
X <- model.matrix(lm(Y ~ Dose.Group * Time.Group + 0, data = table_XY))
dim(X)
```

```
## [1] 64  3
```

```
colnames(X)
```

```
## [1] "Dose.Group"          "Time.Group"          "Dose.Group:Time.Group"
```

```
Y <- scale(Y)
Y[1:3, 1:5]
```

```
##      A_43_P17008 A_43_P19369 A_43_P17618 A_42_P726191 A_43_P19193
## ID202  0.1954138  0.8226673  0.1616244  0.2754313  0.2470066
## ID203  1.0662297  0.9497040  1.4648235  0.2782059  0.6740362
## ID204  0.1219652 -0.1276890 -0.3488965  0.7761968 -0.1385738
```

On a bien les coefficients pour chaque gène (1500) et chaque paramètre du modèle

3) Analyse des résidus du modèle ANOVA

Pour vérifier la validité du modèle multivarié et faire une sélection stable de gènes , examinons les résidus et leurs propriétés

- 1) **Hypothèse d'absence d'autocorrélation** : Les résidus doivent être indépendants entre eux, équivalents à un bruit blanc..

```
# residus = fit$residuals
residus = lm(as.matrix(Y) ~ X - 1)$residuals
dim(residus)
```

```
## [1] 64 1500
```

```
pvalue = whitening_test(residus)
print(pvalue)
```

```
## [1] 1.199177e-36
```

```
dim(cov(residus))
```

```
## [1] 1500 1500
```

La p-valeur du test de blanchiment est très faible, elle affiche 0 car elle est inférieure à 10^{-301} donc R l'enregistre comme nulle. Cela indique une extrême corrélation des résidus et que l'hypothèse d'absence d'autocorrélation est rejetée.


```
## convergence possible : optim renvoie un code = 1
## Warning in arima(residuals[i, ], order = c(pAR, 0, qMA)): problème de
## convergence possible : optim renvoie un code = 1
## Warning in arima(residuals[i, ], order = c(pAR, 0, qMA)): problème de
## convergence possible : optim renvoie un code = 1
## Warning in arima(residuals[i, ], order = c(pAR, 0, qMA)): problème de
## convergence possible : optim renvoie un code = 1
```

```
res_choice
```

```
##          Pvalue      Decision
## AR1          0 NO WHITE NOISE
## nonparam 0.996    WHITE NOISE
## ARMA 1 1      0 NO WHITE NOISE
```

```
print(res_choice$Pvalue)
```

```
## [1] "0"      "0.996" "0"
```

On observe que la p-valeur la plus élevée est obtenue avec le modèle de Toeplitz non paramétrique, suggérant qu'il capture mieux la structure de dépendance des résidus. On va donc utiliser cette méthode pour le blanchiment des résidus.

Vérifions la différence des deux p-valeurs entre le modèle AR(1) et le modèle Toeplitz non paramétrique.

```
# Whitening AR(1)
sqrt_inv_sigma_AR1 <- whitening(residus, "AR1", pAR = 1, qMA = 0)

# Whitening Toeplitz Non-paramétrique
sqrt_inv_sigma_nonparam <- whitening(residus, "nonparam", pAR = 1, qMA = 0)

# Résidus blanchis avec Toeplitz non-param
residus_nonparam <- residus %*% as.matrix(sqrt_inv_sigma_nonparam)

# Résidus blanchis AR(1)
residus_AR1 <- residus %*% as.matrix(sqrt_inv_sigma_AR1)

pvalue_non_param = whitening_test(residus_nonparam)
print(pvalue_non_param)
```

```
## [1] 0.9956886
```

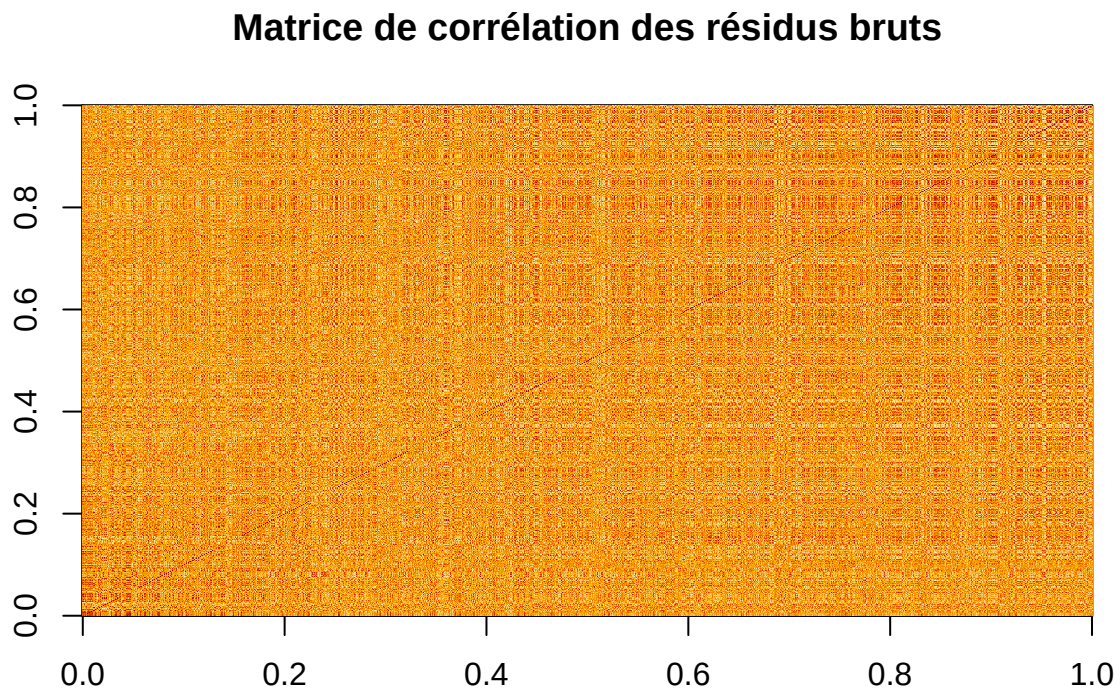
```
pvalue_AR1 = whitening_test(residus_AR1)
print(pvalue_AR1)
```

```
## [1] 1.109013e-30
```

On voit que la p-valeur du modèle AR(1) est très faible de l'ordre de 10^{-30} , ce qui est très loin d'une p-valeur acceptable.

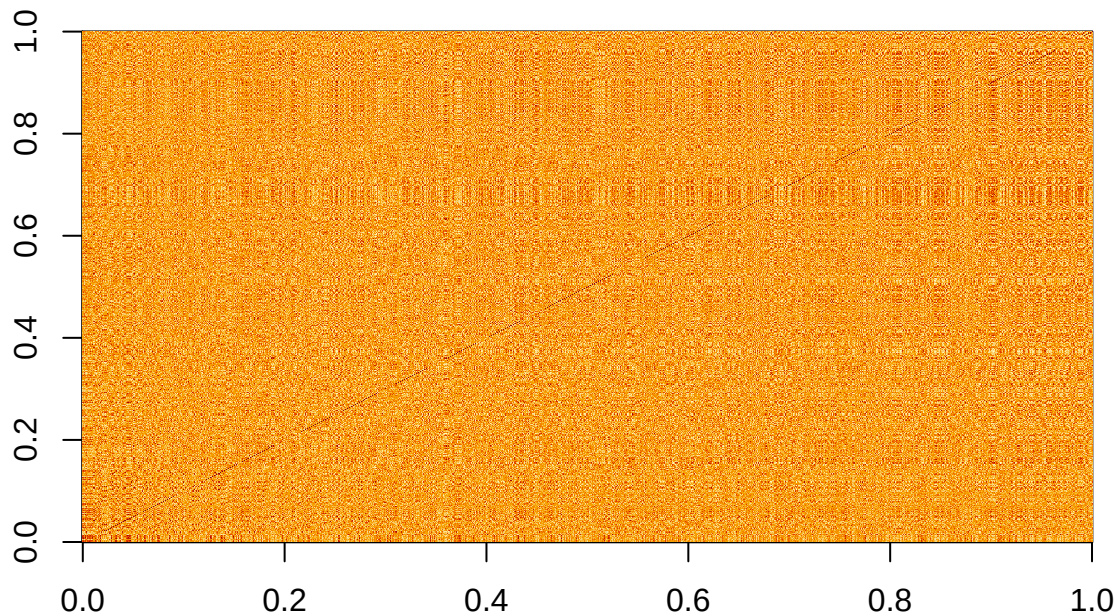
5) Visualisation de l'effet du blanchiment par la matrice de corrélation des résidus

```
image(cor(residus), main = "Matrice de corrélation des résidus bruts")
```



```
cor_residus_blanchis = cor(residus %*% as.matrix(sqrt_inv_sigma_nonparam))  
image(cor_residus_blanchis, main = "Matrice de corrélation des résidus blanchis avec Toeplitz non-param")
```

Matrice de corrélation des résidus blanchis avec Toeplitz non-paramét



La matrice de corrélation des résidus bruts montre une structure de dépendance marquée, avec des blocs de corrélations élevées entre certains gènes. Le blanchiment a atténué cette structure corrélée (moins de blocs foncés de dépendance) pour obtenir des résidus plus proches d'un bruit blanc multivarié.

6) Sélection des gènes influencés par les facteurs expérimentaux par la Stability Selection

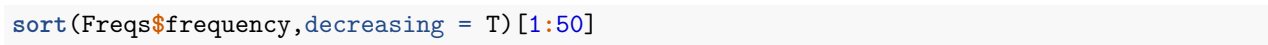
Maintenant que nous avons stabilisé les résidus via le blanchiment, nous pouvons procéder à la sélection des gènes influencés par les facteurs dose, temps et leur interaction. Nous allons utiliser la Stability Selection, une méthode robuste pour identifier les variables pertinentes dans un contexte de haute dimension. L'idée est de répéter plusieurs fois le processus de sélection sur des sous-échantillons des données et de calculer la fréquence de sélection de chaque gène apparu dans les couples variable-réponse.

Cette méthode peut-être très lourde car elle nécessite le calcul de la matrice de Kronecker. Ici...

```
# require(doMC)
# registerDoMC(cores = 2)
# Freqs = variable_selection(Y, X, sqrt_inv_sigma_nonparam, nb_repli = 100, parallel = TRUE, nb.cores = 2)
# save(Freqs, file = 'Freqs_liver_toxicity_TOEPLITZ_nbreplis_100.Rdata')
load('Freqs_liver_toxicity_TOEPLITZ_nbreplis_100.Rdata')
```

```
colnames(Freqs) <- c('Names_of_Y', 'Names_of_X', 'frequency')
```

```
plot(sort(Freqs$frequency, decreasing = T), type = 's',
      main = "Fréquence de sélection des couples variables-réponse",
      xlab = "Couples Variables-Réponse (triées par fréquence)", ylab = "Fréquence de sélection")
```



```
seuil <- 0.93
indices = which(Freqs$frequency >= seuil)
length(indices)
```

14

```
selected_genes <- unique(Freqs[indices, ]$Names_of_Y)
length(selected_genes)
```

```
## [1] 1146
```

1146 gènes sont sélectionnés avec ce seuil.

E) Visualisation et interprétation des résultats

Pour avoir des visuels lisibles, on va regarder les 10 gènes les plus influencés selon chaque effet (dose, temps, interaction) parmi les gènes sélectionnés. Nous avons besoin de réestimer les coefficients pour ces gènes car le LASSO introduit un biais.

Reestimation des coefficients pour les variables sélectionnés

```
# Étape 1 : construction des vecteurs Yvec et Xvec
# Yvec <- as.numeric(Y %%% sqrt_inv_sigma_nonparam)

# Xvec <- kronecker(t(sqrt_inv_sigma_nonparam), X)

## Étape 2 : sélection des colonnes correspondant aux variables retenues
# Xvec_sel <- Xvec[, indices]

## Étape 3 : estimation des coefficients (solution des moindres carrés)
# B_sel_hat <- solve(t(Xvec_sel) %%% Xvec_sel, t(Xvec_sel) %%% Yvec)

## Étape 4 : stockage dans Freqs
# p <- ncol(X)
# q <- ncol(Y)
# Freqs$estim <- rep(0, p * q)
# Freqs$estim[indices] <- as.vector(B_sel_hat)
# Save Freqs with estim
# save(Freqs, file = 'Freqs_liver_toxicity_TOEPLITZ_nbreplic_100_with_estim.Rdata')
load('Freqs_liver_toxicity_TOEPLITZ_nbreplic_100_with_estim.Rdata')
```

Sélection des 10 gènes les plus influencés selon chaque effet (par les valeurs réestimés)

```
library(dplyr)
```

```
##
```

```
## Attachement du package : 'dplyr'
```

```
## Les objets suivants sont masqués depuis 'package:stats':
```

```
##
```

```
## filter, lag
```

```
## Les objets suivants sont masqués depuis 'package:base':
##
## intersect, setdiff, setequal, union
```

```
# Nettoyage des noms
Freqs$Names_of_X <- gsub("Dose.Group", "Dose", Freqs$Names_of_X)
Freqs$Names_of_X <- gsub("Time.Group", "Time", Freqs$Names_of_X)
Freqs$Names_of_X <- gsub(":", " * ", Freqs$Names_of_X)
# Freqs$Gene_Names <- table4$gene.title[Freqs$Names_of_Y == table4$geneBank]

# --- 1) SELECTION des 10 plus influents POUR TIME
Freqs_time <- Freqs %>% filter(grepl("^Time$", Names_of_X), frequency >= seuil)
top10_time <- Freqs_time %>% arrange(desc(abs(estim))) %>% head(10)

# --- 2) SELECTION des 10 plus influents POUR DOSE
Freqs_dose <- Freqs %>% filter(grepl("^Dose$", Names_of_X), frequency >= seuil)
top10_dose <- Freqs_dose %>% arrange(desc(abs(estim))) %>% head(10)

# --- 3) SELECTION des 10 plus influents POUR DOSE×TIME
Freqs_inter <- Freqs %>% filter(grepl("Dose \\* Time", Names_of_X), frequency >= seuil)
top10_inter <- Freqs_inter %>% arrange(desc(abs(estim))) %>% head(10)

top10_time$Effet <- top10_time$Names_of_X
top10_dose$Effet <- top10_dose$Names_of_X
top10_inter$Effet <- top10_inter$Names_of_X

top10_dose
```

##	Names_of_Y	Names_of_X	frequency	estim	Effet
## 1	A_43_P11891	Dose	1	-0.001147073	Dose
## 2	A_42_P607568	Dose	1	-0.001130406	Dose
## 3	A_43_P11456	Dose	1	-0.001120078	Dose
## 4	A_42_P713290	Dose	1	0.001102595	Dose
## 5	A_43_P21061	Dose	1	-0.001097314	Dose
## 6	A_43_P10088	Dose	1	-0.001057306	Dose
## 7	A_42_P642415	Dose	1	0.001050669	Dose
## 8	A_43_P11468	Dose	1	0.001041830	Dose
## 9	A_42_P678904	Dose	1	-0.001034764	Dose
## 10	A_43_P14713	Dose	1	0.001033069	Dose

top10_time

##	Names_of_Y	Names_of_X	frequency	estim	Effet
## 1	A_42_P531540	Time	1	0.03318873	Time
## 2	A_42_P839593	Time	1	0.03205478	Time
## 3	A_43_P12724	Time	1	0.03169536	Time
## 4	A_42_P501838	Time	1	0.03054077	Time
## 5	A_42_P807568	Time	1	0.03001399	Time
## 6	A_42_P595591	Time	1	-0.02911611	Time
## 7	A_42_P459204	Time	1	0.02896455	Time
## 8	A_42_P678279	Time	1	0.02893702	Time
## 9	A_43_P13978	Time	1	-0.02891873	Time
## 10	A_43_P18047	Time	1	-0.02861075	Time

```
top10_inter
```

##	Names_of_Y	Names_of_X	frequency	estim	Effet
## 1	A_42_P843603	Dose * Time	1	4.790552e-05	Dose * Time
## 2	A_43_P21061	Dose * Time	1	4.656893e-05	Dose * Time
## 3	A_42_P576823	Dose * Time	1	4.562662e-05	Dose * Time
## 4	A_42_P715976	Dose * Time	1	4.441308e-05	Dose * Time
## 5	A_43_P15363	Dose * Time	1	4.373416e-05	Dose * Time
## 6	A_42_P685287	Dose * Time	1	4.276028e-05	Dose * Time
## 7	A_42_P477643	Dose * Time	1	4.235883e-05	Dose * Time
## 8	A_43_P17093	Dose * Time	1	4.206627e-05	Dose * Time
## 9	A_43_P12413	Dose * Time	1	4.170191e-05	Dose * Time
## 10	A_42_P772385	Dose * Time	1	4.143736e-05	Dose * Time

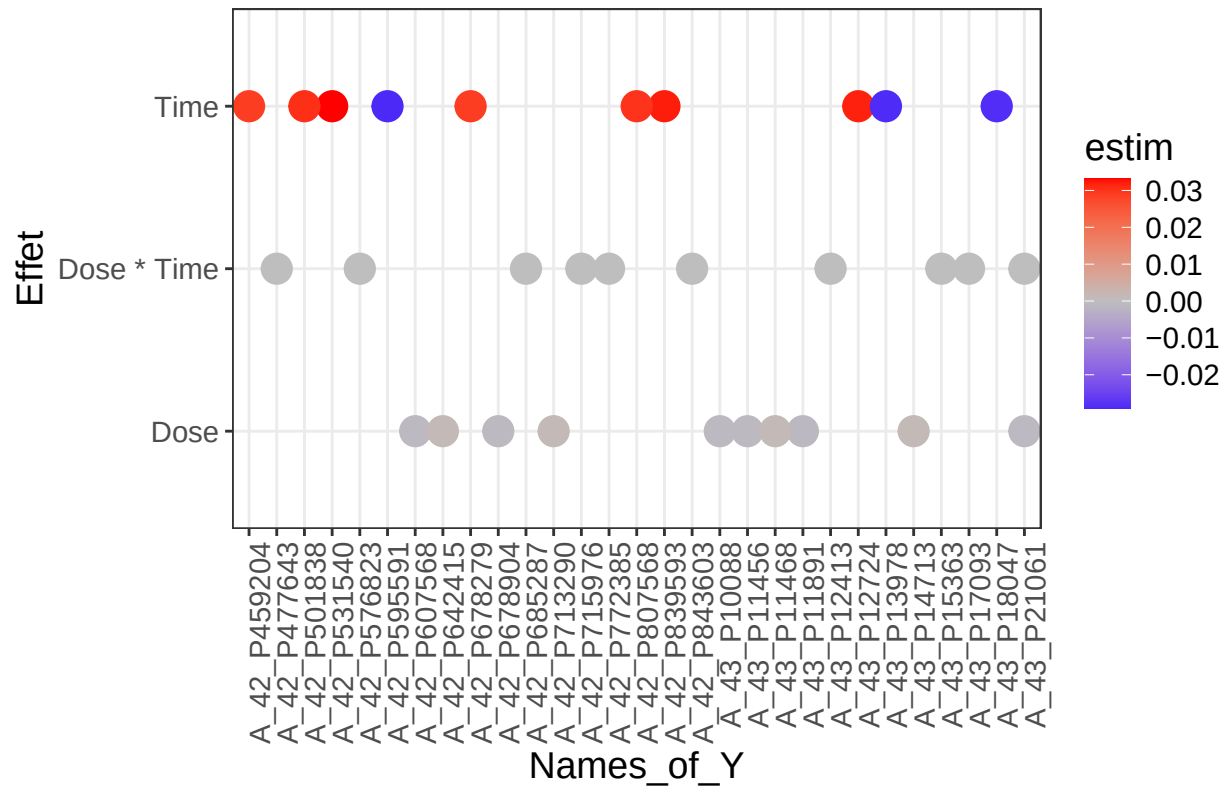
Heatmap des effets des facteurs sur 10 gènes parmi les gènes sélectionnés

```
library(ggplot2)
top30 <- rbind(top10_dose, top10_time, top10_inter)
#i want the names of the genes, not the id, there is the names in table4

gr = ggplot(top30, aes(
  x = Names_of_Y,
  y = Effet,
  color = estim
)) +
  geom_point(size = 5) +
  scale_color_gradient2(low = "blue", mid = "grey", high = "red") +
  theme_bw() +
  theme(
    axis.text.x = element_text(angle=90, hjust=1),
    text = element_text(size=14)
  ) +
  ggtitle("Top 10 gènes les plus influents par effet")

print(gr)
```

Top 10 gènes les plus influents par effet



```
#save the plot
# ggsave("heatmap_effects_selected_genes.png", plot = last_plot())
```

On observe que certains gènes sont fortement influencés par le facteur temps

Boxplots de 10 gènes parmi les gènes sélectionnés

```
library(reshape2)
library(ggplot2)
library(dplyr)

# -----
# 1) Créer une liste avec les 3 catégories
# -----
top_list <- list(
  "Time"      = top10_time$Names_of_Y,
  "Dose"      = top10_dose$Names_of_Y,
  "Dose * Time" = top10_inter$Names_of_Y
)

# -----
# 2) Boucle pour générer les 3 images
# -----
```

```

for (effet in names(top_list)) {

  # Gènes à tracer pour cet effet
  genes_to_plot <- top_list[[effet]]

  # Construction du tableau réduit
  table_red <- as.data.frame(Y[, genes_to_plot, drop = FALSE])

  # Ajout des colonnes Dose et Time
  table_red$Dose <- factor(table_XY$Dose.Group)
  table_red$Time <- factor(table_XY$Time.Group)

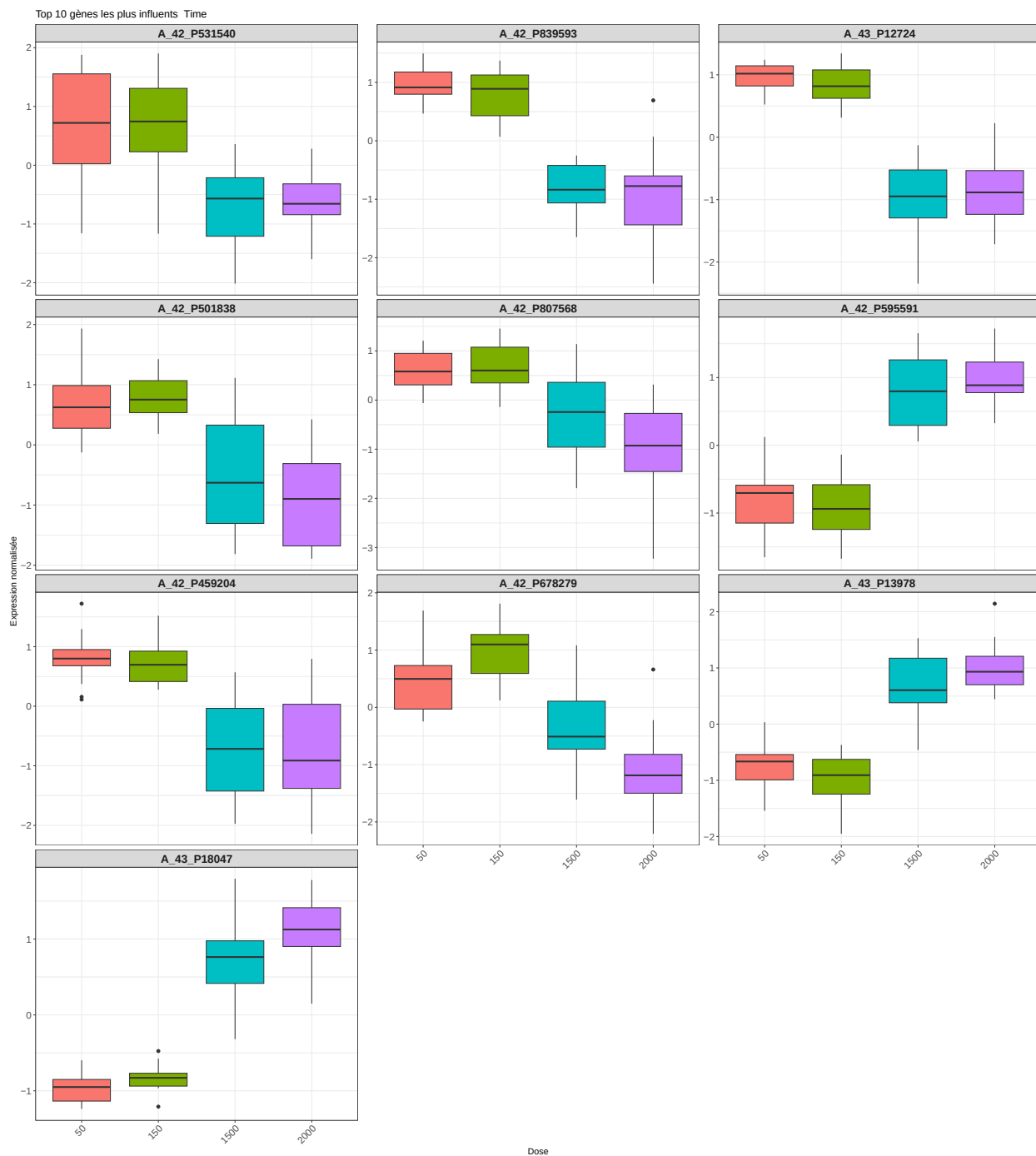
  # Passage au format long
  table_long <- melt(table_red, id.vars = c("Dose", "Time"))

  # Création du graphique
  g <- ggplot(table_long, aes(x = Dose, y = value, fill = Dose)) +
    geom_boxplot() +
    facet_wrap(~variable, scales = "free_y", ncol = 3) +
    theme_bw() +
    theme(
      axis.text.x = element_text(size=12, angle=45, hjust=1),
      axis.text.y = element_text(size=12),
      strip.text = element_text(size=14, face="bold"),
      legend.position = "none"
    ) +
    ylab("Expression normalisée") +
    xlab("Dose") +
    ggtitle(paste("Top 10 gènes les plus influents ", effet))

  print(g)

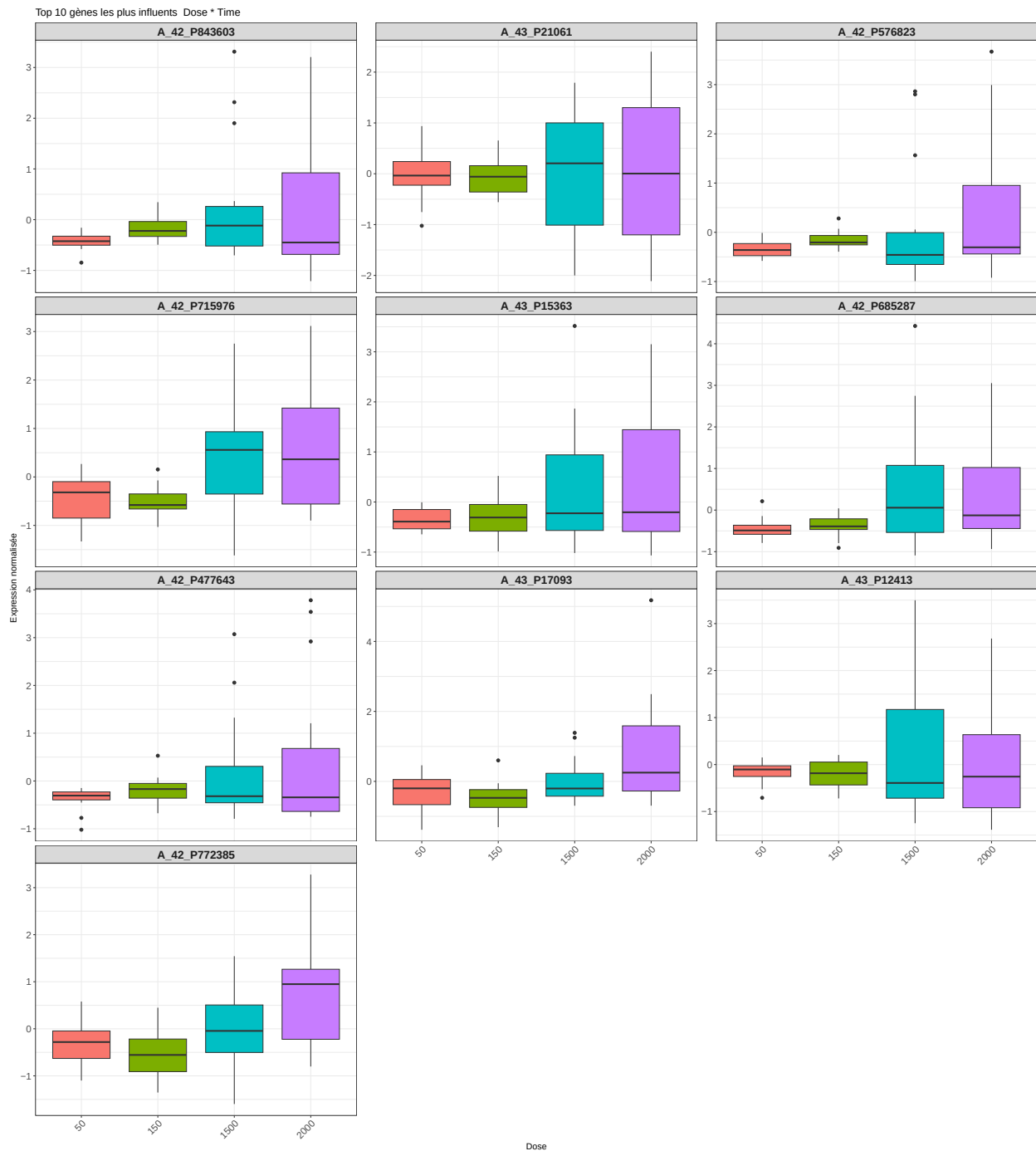
  # Sauvegarde de l'image
  filename <- paste0("Top10_genes_", gsub("\\*", "_", effet), ".png")
  ggsave(filename, plot = g, width = 12, height = 10)
}

```



Top 10 gènes les plus influents Dose





On voit que les gènes sélectionnés montrent des variations d'expression significatives en fonction des facteurs dose, temps et leur interaction.

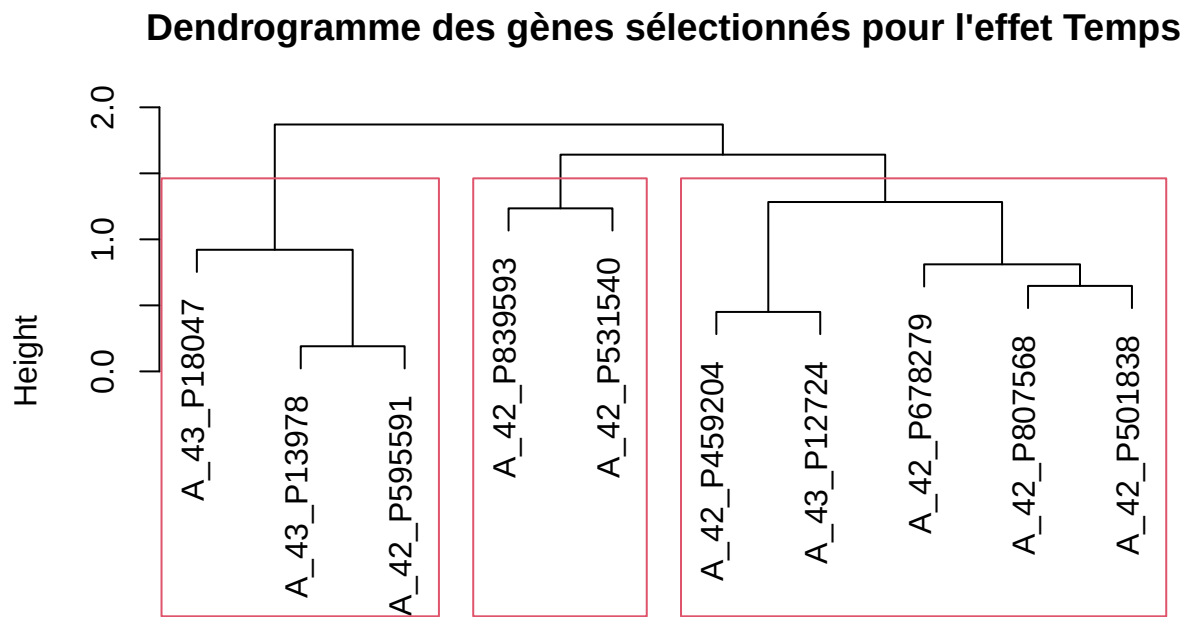
Dendrogramme de 10 gènes parmi les gènes sélectionnés

```
liste_genes_selected <- unique(top10_time$Names_of_Y)
length(liste_genes_selected)
```

```
## [1] 10
```

```
residus_red <- residus[, colnames(residus) %in% liste_genes_selected]
```

```
hc <- hclust(dist(1 - abs(cor(residus_red))))
plot(hc, main = "Dendrogramme des gènes sélectionnés pour l'effet Temps")
rect.hclust(hc, k = 3)
```



```
dist(1 - abs(cor(residus_red)))
hclust (*, "complete")
```

```
##save the plot as png
dev.copy(png, "Dendrogram_selected_genes.png", width = 800, height = 600)
```

```
## png
## 3
```

```
dev.off()
```

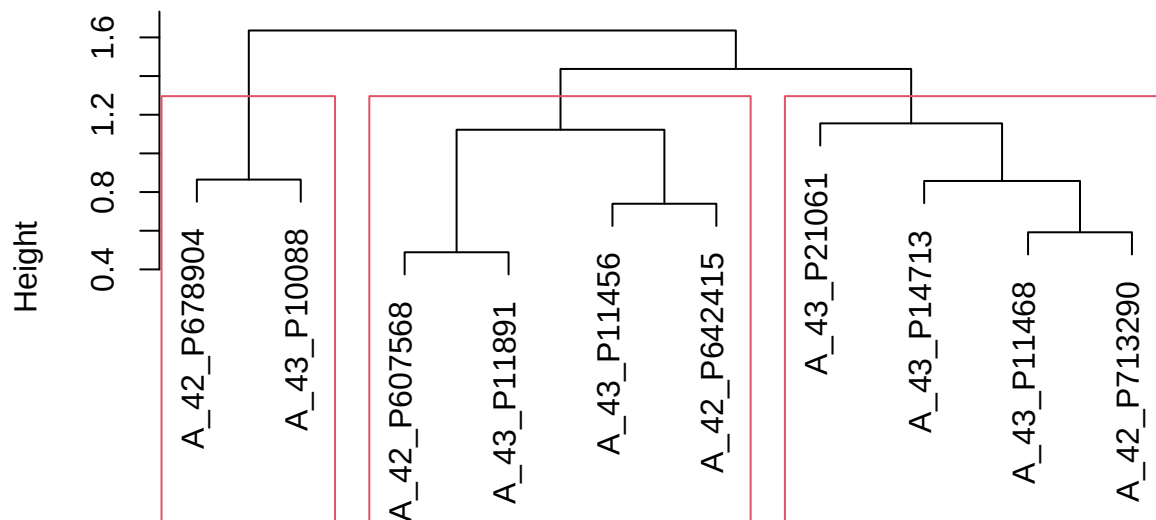
```
## pdf
## 2
```

```
plot.new()
liste_genes_selected <- unique(top10_dose$Names_of_Y)
length(liste_genes_selected)
```

```
## [1] 10
```

```
residus_red <- residus[, colnames(residus) %in% liste_genes_selected]
hc <- hclust(dist(1 - abs(cor(residus_red))))
plot(hc, main = "Dendrogramme des gènes sélectionnés pour l'effet Dose")
rect.hclust(hc, k = 3)
```

Dendrogramme des gènes sélectionnés pour l'effet Dose



```
dist(1 - abs(cor(residus_red)))
hclust (*, "complete")
```

```
##save the plot as png
dev.copy(png, "Dendrogram_selected_genes_dose.png", width = 800, height = 600)
```

```
## png
## 3
```

```
dev.off()
```

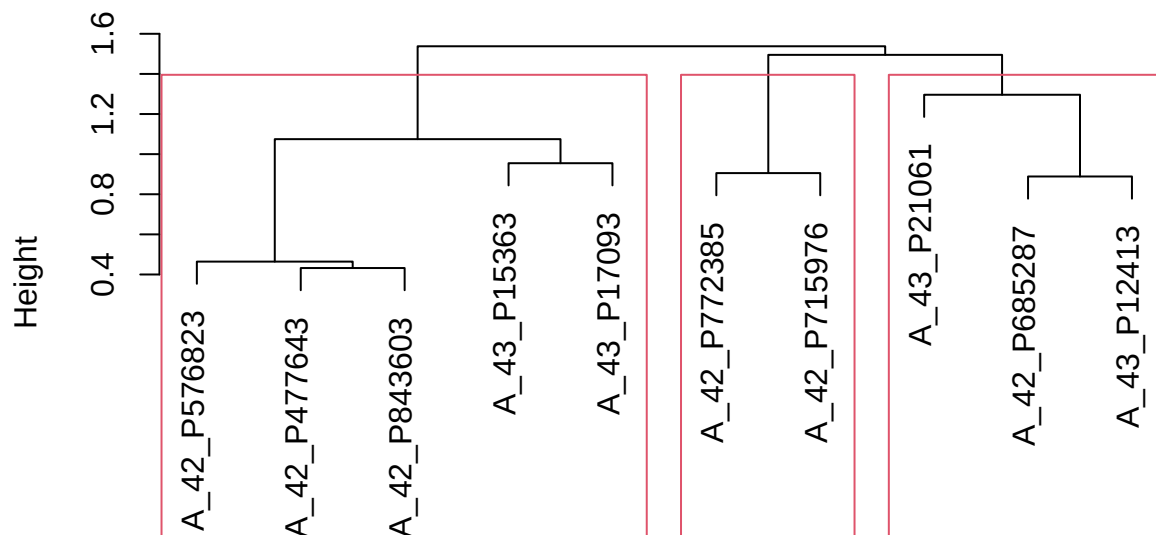
```
## pdf
## 2
```

```
plot.new()
liste_genes_selected <- unique(top10_inter$Names_of_Y)
length(liste_genes_selected)
```

```
## [1] 10
```

```
residus_red <- residus[, colnames(residus) %in% liste_genes_selected]
hc <- hclust(dist(1 - abs(cor(residus_red))))
plot(hc, main = "Dendrogramme des gènes sélectionnés pour l'effet Dose x Temps")
rect.hclust(hc, k = 3)
```

Dendrogramme des gènes sélectionnés pour l'effet Dose x Temps



```
dist(1 - abs(cor(residus_red)))
hclust(*, "complete")
```

```
##save the plot as png
dev.copy(png, "Dendrogram_selected_genes_interaction.png", width = 800, height = 600)
```

```
## png
## 3
```

```
dev.off()
```

```
## pdf
## 2
```

On observe des clusters de gènes avec des profils d'expression similaires parmi les gènes sélectionnés, ce qui peut indiquer des régulations transcriptionnelles communes en réponse aux facteurs dose et temps. Cela suggère que ces gènes pourraient être impliqués dans des voies biologiques similaires affectées par le paracétamol.

F) Difficultés rencontrées

La principale difficulté a été la lourdeur computationnelle de la Stability Selection, en particulier le calcul de la matrice de Kronecker. Pour alléger les calculs, nous avons réduit la dimensionnalité en sélectionnant les 1500 gènes les plus variables. Le choix du modèle de dépendance des résidus a également été crucial pour obtenir des résidus blanchis satisfaisants. Dans un premier temps, à cause d'une autre approche de modélisation, nous avons eu des p-valeurs de blanchiments très faibles pour les 3 familles de modèles, ce qui nous a permis d'explorer la modélisation par les matrices de covariances par blocs.

NB: Pour la fonction `whitening_choice` de `MultiVarSel`, les p-valeurs affichées peuvent s'afficher ZERO alors qu'elles sont supérieur au seuil enregistrable de R

G) Conclusion

Ce projet fut très formateur et a permis d'appliquer des techniques avancées de modélisation statistique multivariée à un problème réel de toxicité médicamenteuse. La mise en situation dans un environnement complexe par la contrainte des ressources disponibles a été un défi intéressant à relever.